PukSquash Developer Manual

Registration Login and Access Control

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

This functionality creates a Firebase email and password account, sends email verification, signs the user in and supplies verified identity to Laravel. The Firebase account owns the credentials. The PukSquash User record owns the application name, student number, role and account status.

SignUpPage calls registerUser and waits for verification. It reloads the Firebase user every three seconds and stops the timer when the component unmounts. GET /api/me creates the minimal Data Connect profile if it is missing. LoginPage signs in, checks emailVerified and refreshes the account. Vue route guards guide navigation; Laravel verifies protected requests independently.

1 1 Files and Responsibilities

resources/js/pages/SignUpPage.vue and LoginPage.vue — forms and verification flow

resources/js/features/auth/authService.js — registration, login and logout

resources/js/composables/useAuth.js and useAccount.js — shared identity and profile state

app/Services/FirebaseAuthService.php — Firebase server client

app/Http/Middleware/FirebaseAuth.php and AdminMiddleware.php — access checks

app/Http/Controllers/Api/UserController.php — GET /api/me

1 2 API Contract

Firebase SDK calls handle registration and login. GET /api/me reads or creates the application profile. There is no Laravel password-login endpoint in this edition.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Firebase registration implementation

resources/js/features/auth/authService.js | source line 49 onward; blank and comment lines omitted

```
export async function registerUser(email, password) {
    startLoading();
    try {
        const result = await createUserWithEmailAndPassword(
            auth,
            normalizeEmail(email),
            password
        );
        await sendEmailVerification(result.user);
        return result;
    } catch (error) {
        throw new Error(friendlyAuthError(error));
    } finally {
        stopLoading();
    }
}
```

Explanation

The authentication service uses the Firebase auth instance imported from services/firebase.js, creates the account and sends the verification email. It converts common Firebase error codes to readable messages. The shared loading indicator starts before the request and stops in finally, including on failure. Account creation and email verification are separate steps.

Physical Code Explanation

1 The auth instance is statically imported in this revision. startLoading begins shared feedback, and stopLoading in finally resets it. Configure Firebase before loading the authentication pages.

2 createUserWithEmailAndPassword receives the trimmed email and original password. Passwords are handled by Firebase, not stored in the Data Connect User entity.

3 sendEmailVerification sends the verification link. If sending fails after account creation, the account may already exist; retry and recovery must account for that partial outcome.

4 loginUser uses signInWithEmailAndPassword. logoutUser delegates to Firebase signOut. useAuth observes onAuthStateChanged for application-wide session state.

The signup form prevents repeated submission while waiting for verification. A verified signup proceeds to the booking page. The baseline has no registration payment gate. Payment-dependent registration is covered separately as an extension.

3 Profile creation after authentication

app/Http/Controllers/Api/UserController.php | source line 23 onward; blank and comment lines omitted

```
public function me(Request $request): JsonResponse
{
    $firebaseUid = $request->attributes->get('firebase_uid');
    $email       = $request->attributes->get('firebase_email');
    $user = $this->users->findByFirebaseUid($firebaseUid);
    if (! $user) {
        $studentNumber = $email ? explode('@', $email)[0] : null;
        $this->users->upsert([
            'firebaseUid' => $firebaseUid,
            'fullName'    => explode('@', $email)[0],
            'email'       => $email,
            'studentNumber' => $studentNumber,
        ]);
        $user = $this->users->findByFirebaseUid($firebaseUid);
    }
    return response()->json($user?->toArray());
}
```

Explanation

FirebaseAuth verifies the bearer token, resolves its subject UID and fetches the Firebase user. It places the UID, email and verification flag in request attributes. GET /api/me is deliberately allowed before email verification so the minimal profile can be created; other protected API requests reject an unverified account.

Physical Code Explanation

1 The controller reads firebase_uid and firebase_email from middleware attributes, not from browser input.

2 findByFirebaseUid links the Firebase identity to its application profile. If absent, the email prefix supplies a temporary display name and student number.

3 UserRepository::upsert defaults role to USER and status to ACTIVE, then executes the UpsertUser insert mutation. The controller reads the created record and returns its DTO array.

4 AdminMiddleware additionally requires an ACTIVE profile with role ADMIN. Coach controllers explicitly require an ACTIVE COACH. Client-side requiresRole metadata controls the screen, not the authority to call the API.

The method named UpsertUser is implemented with user_insert and duplicate-error handling in this baseline. It is not a general update-on-conflict mutation. Keep retries and cache invalidation consistent with that behaviour.

4 Verification and Maintenance

Verification checklist

1 Register a new test address: expect a Firebase account and verification email; verify the link and confirm navigation and GET /api/me.

2 Try a duplicate email, invalid email, weak password, incorrect password and unavailable network. Confirm an understandable error and a usable retry path.

3 Call a protected endpoint without a token or with an invalid token: expect 401. Call with an unverified account: expect 403, except GET /api/me.

4 Use USER, COACH and ADMIN test accounts against the actual protected APIs. A hidden link or blocked browser route alone is not an authorisation test.

5 Navigate away during verification and check that polling stops. Log out, then attempt to reuse the authenticated page and confirm that a new protected request requires authentication.

Implementation boundaries

The specifications require university email eligibility. The baseline trims email but does not implement an NWU domain allowlist. Add a server-enforced eligibility policy before claiming NWU-only registration.

FirebaseAuth in this baseline does not globally reject suspended or inactive application profiles; some role-specific controllers do. This is a known access-control gap for general booking routes. Password-reset and Google sign-in screens from the example manual are not implemented here.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

https://firebase.google.com/docs/auth/web/password-auth

https://firebase.google.com/docs/auth/admin/verify-id-tokens